

Tarea 2: Simulación Molecular Avanzada

Unidad 2 — Modelado, Simulación e IA en Nanotecnología

Este notebook contiene tres tareas que aplican los conceptos de Dinámica Molecular (MD) y Monte Carlo (MC) aprendidos en la Unidad 2:

1. **Tarea 1: Efecto de Temperatura** — Simulación MD a múltiples temperaturas
2. **Tarea 2: Comparación MD vs MC** — Implementación y comparación de ambos métodos
3. **Tarea 3: Nanofabricación** — Simulación MC de crecimiento de monocapa

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
import time
import warnings
warnings.filterwarnings('ignore')

# Configuración global de matplotlib
plt.rcParams['figure.figsize'] = (14, 10)
plt.rcParams['font.size'] = 12
plt.rcParams['axes.grid'] = True
plt.rcParams['grid.alpha'] = 0.3

print("✅ Librerías importadas correctamente")
```

✅ Librerías importadas correctamente

Funciones Base: Potencial Lennard-Jones, PBC y Dinámica Molecular

Las siguientes funciones implementan el motor de simulación que será utilizado en las tres tareas. Se basa en el potencial de Lennard-Jones con condiciones periódicas de frontera (PBC) en 2D.

```
In [2]: # =====
# FUNCIONES BASE DE SIMULACIÓN
# =====

def lj_potential(r, epsilon=1.0, sigma=1.0):
    """Potencial de Lennard-Jones"""
    sr6 = (sigma/r)**6
    return 4 * epsilon * (sr6**2 - sr6)
```

```

def lj_force(r, epsilon=1.0, sigma=1.0):
    """Fuerza de Lennard-Jones (magnitud, en dirección radial)"""
    sr6 = (sigma/r)**6
    return 24 * epsilon * (2*sr6**2 - sr6) / r

def apply_pbc(positions, box_size):
    """Aplica condiciones periódicas de frontera"""
    return positions - np.floor(positions / box_size) * box_size

def minimum_image(r_ij, box_size):
    """Convención de imagen mínima"""
    return r_ij - np.round(r_ij / box_size) * box_size

def compute_forces_energy(positions, box_size, epsilon=1.0, sigma=1.0, r_cut=2.5):
    """Calcula fuerzas y energía potencial del sistema usando LJ con PBC."""
    n = len(positions)
    forces = np.zeros_like(positions)
    potential_energy = 0.0

    for i in range(n):
        for j in range(i+1, n):
            r_ij = positions[j] - positions[i]
            r_ij = minimum_image(r_ij, box_size)
            r = np.linalg.norm(r_ij)
            if r < r_cut * sigma and r > 0.8 * sigma:
                f_mag = lj_force(r, epsilon, sigma)
                f_vec = f_mag * r_ij / r
                forces[i] += f_vec
                forces[j] -= f_vec
                potential_energy += lj_potential(r, epsilon, sigma)
    return forces, potential_energy

def init_positions_grid(n_particles, box_size):
    """Inicializa partículas en una red cuadrada."""
    n_side = int(np.ceil(np.sqrt(n_particles)))
    spacing = box_size / n_side
    positions = []
    for i in range(n_side):
        for j in range(n_side):
            if len(positions) < n_particles:
                positions.append([spacing*(i+0.5), spacing*(j+0.5)])
    return np.array(positions)

def init_velocities(n_particles, temperature, mass=1.0):
    """Inicializa velocidades con distribución Maxwell-Boltzmann."""
    k_B = 1.0 # En unidades reducidas
    sigma_v = np.sqrt(k_B * temperature / mass)
    velocities = np.random.normal(0, sigma_v, (n_particles, 2))
    # Remover movimiento del centro de masa
    velocities -= velocities.mean(axis=0)
    return velocities

def rescale_velocities(velocities, target_temp, mass=1.0):
    """Reescala velocidades para alcanzar temperatura objetivo."""
    k_B = 1.0

```

```

n = len(velocities)
KE = 0.5 * mass * np.sum(velocities**2)
current_temp = KE / (n * k_B) # 2D: 2 grados de libertad por partícula / 2
if current_temp > 1e-10:
    scale = np.sqrt(target_temp / current_temp)
    velocities *= scale
return velocities

def run_md(n_particles, box_size, temperature, n_steps, dt=0.002,
           n_equilibration=1000, rescale_interval=50, epsilon=1.0, sigma=1.0, mass=
           """Ejecuta simulación de Dinámica Molecular con Velocity-Verlet y termostatación
           k_B = 1.0
           positions = init_positions_grid(n_particles, box_size)
           velocities = init_velocities(n_particles, temperature, mass)

           forces, pe = compute_forces_energy(positions, box_size, epsilon, sigma)

           energies_pot = []
           energies_kin = []
           energies_tot = []
           temperatures_list = []
           all_positions = []

           for step in range(n_steps):
               # Velocity-Verlet: actualizar posiciones
               positions += velocities * dt + 0.5 * forces / mass * dt**2
               positions = apply_pbc(positions, box_size)

               # Nuevas fuerzas
               new_forces, pe = compute_forces_energy(positions, box_size, epsilon, sigma)

               # Velocity-Verlet: actualizar velocidades
               velocities += 0.5 * (forces + new_forces) / mass * dt
               forces = new_forces

               # Limitar velocidades extremas para estabilidad
               v_max = 5.0 * np.sqrt(temperature)
               speeds = np.linalg.norm(velocities, axis=1)
               mask = speeds > v_max
               if mask.any():
                   velocities[mask] *= (v_max / speeds[mask, np.newaxis])

               # Termostatación durante equilibración
               if step < n_equilibration and step % rescale_interval == 0:
                   velocities = rescale_velocities(velocities, temperature, mass)

               # Calcular energía cinética
               ke = 0.5 * mass * np.sum(velocities**2)
               current_temp = ke / (n_particles * k_B)

               energies_pot.append(pe)
               energies_kin.append(ke)
               energies_tot.append(pe + ke)
               temperatures_list.append(current_temp)

               if step % 100 == 0:

```

```

        all_positions.append(positions.copy())

    return {
        'energies_pot': np.array(energies_pot),
        'energies_kin': np.array(energies_kin),
        'energies_tot': np.array(energies_tot),
        'temperatures': np.array(temperatures_list),
        'positions_snapshots': all_positions,
        'final_positions': positions.copy(),
        'final_velocities': velocities.copy(),
        'n_equilibration': n_equilibration
    }

def compute_rdf(positions, box_size, n_bins=100, r_max=None):
    """Calcula la función de distribución radial g(r)."""
    if r_max is None:
        r_max = box_size / 2.0
    n = len(positions)
    dr = r_max / n_bins
    hist = np.zeros(n_bins)

    for i in range(n):
        for j in range(i+1, n):
            r_ij = positions[j] - positions[i]
            r_ij = minimum_image(r_ij, box_size)
            r = np.linalg.norm(r_ij)
            if r < r_max:
                bin_idx = int(r / dr)
                if bin_idx < n_bins:
                    hist[bin_idx] += 2 # Contar ambas direcciones

    # Normalizar
    r_values = np.linspace(dr/2, r_max - dr/2, n_bins)
    area = box_size**2
    rho = n / area
    for k in range(n_bins):
        r_lo = k * dr
        r_hi = (k+1) * dr
        shell_area = np.pi * (r_hi**2 - r_lo**2)
        ideal = rho * shell_area
        if ideal > 0:
            hist[k] /= (n * ideal)

    return r_values, hist

def compute_msd(positions_list):
    """Calcula el desplazamiento cuadrático medio (MSD)."""
    n_frames = len(positions_list)
    if n_frames < 2:
        return np.array([0.0])
    ref = positions_list[0]
    msd = []
    for frame in positions_list:
        disp = frame - ref
        msd.append(np.mean(np.sum(disp**2, axis=1)))
    return np.array(msd)

```

```
print("✅ Funciones base definidas correctamente")
```

✅ Funciones base definidas correctamente

Tarea 1: Efecto de Temperatura

Ejecutamos la simulación de Dinámica Molecular a $T = 100, 300, 500$ y 700 K (en unidades reducidas: $T^* = 0.5, 1.0, 2.0, 3.5$) para analizar:

- Energía promedio vs T
- Coeficiente de difusión vs T (gráfica de Arrhenius)
- Estructura (RDF) a diferentes temperaturas

```
In [3]: # =====
# TAREA 1: EFECTO DE TEMPERATURA - SIMULACIÓN MD
# =====
np.random.seed(42)

# Parámetros del sistema
n_particles = 16
box_size = 8.0
n_steps = 3000
dt = 0.002
n_eq = 1000

# Temperaturas en unidades reducidas ( $T^* = k_B T / \epsilon$ )
temperatures = [0.5, 1.0, 2.0, 3.5]
temp_labels = ['T*=0.5 (100K)', 'T*=1.0 (300K)', 'T*=2.0 (500K)', 'T*=3.5 (700K)']
colors = ['blue', 'green', 'orange', 'red']

results_t1 = {}

for i, T in enumerate(temperatures):
    print(f"\nSimulando {temp_labels[i]}...")
    result = run_md(n_particles, box_size, T, n_steps, dt, n_eq)
    results_t1[T] = result
    eq_data = result['energies_pot'][n_eq:]
    print(f"  Energía potencial promedio: {eq_data.mean():.3f} ± {eq_data.std():.3f}")
    print(f"  Temperatura promedio: {result['temperatures'][n_eq:].mean():.3f}")

print("\n✅ Simulaciones completadas")
```

Simulando $T^*=0.5$ (100K)...

Energía potencial promedio: -0.665 ± 10.093

Temperatura promedio: 1.521

Simulando $T^*=1.0$ (300K)...

Energía potencial promedio: -1.660 ± 7.287
Temperatura promedio: 1.096

Simulando $T^*=2.0$ (500K)...

Energía potencial promedio: 1.649 ± 11.036
Temperatura promedio: 2.028

Simulando $T^*=3.5$ (700K)...

Energía potencial promedio: 8.494 ± 17.866
Temperatura promedio: 4.850

✓ Simulaciones completadas

1.1 Energía Promedio vs Temperatura

```
In [4]: # Gráfico de Energía promedio vs Temperatura
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Panel 1: Evolución de La energía potencial
ax = axes[0, 0]
for i, T in enumerate(temperatures):
    r = results_t1[T]
    steps = np.arange(len(r['energies_pot']))
    ax.plot(steps, r['energies_pot'], color=colors[i], alpha=0.6, linewidth=0.8, la
    ax.axvline(x=n_eq, color='black', linestyle='--', alpha=0.5)
ax.set_xlabel('Paso MD')
ax.set_ylabel('Energía Potencial')
ax.set_title('Evolución de la Energía Potencial')
ax.legend(fontsize=9)

# Panel 2: Energía promedio vs T
ax = axes[0, 1]
T_vals = []
E_pot_avg = []
E_kin_avg = []
E_tot_avg = []
for T in temperatures:
    r = results_t1[T]
    T_vals.append(r['temperatures'][n_eq:].mean())
    E_pot_avg.append(r['energies_pot'][n_eq:].mean())
    E_kin_avg.append(r['energies_kin'][n_eq:].mean())
    E_tot_avg.append(r['energies_tot'][n_eq:].mean())

ax.plot(T_vals, E_pot_avg, 'bo-', label='Potencial', markersize=8)
ax.plot(T_vals, E_kin_avg, 'rs-', label='Cinética', markersize=8)
ax.plot(T_vals, E_tot_avg, 'g^-', label='Total', markersize=8)
ax.set_xlabel('Temperatura (T*)')
ax.set_ylabel('Energía Promedio')
ax.set_title('Energía vs Temperatura')
ax.legend()

# Panel 3: Distribución de energías
ax = axes[1, 0]
for i, T in enumerate(temperatures):
    r = results_t1[T]
```

```

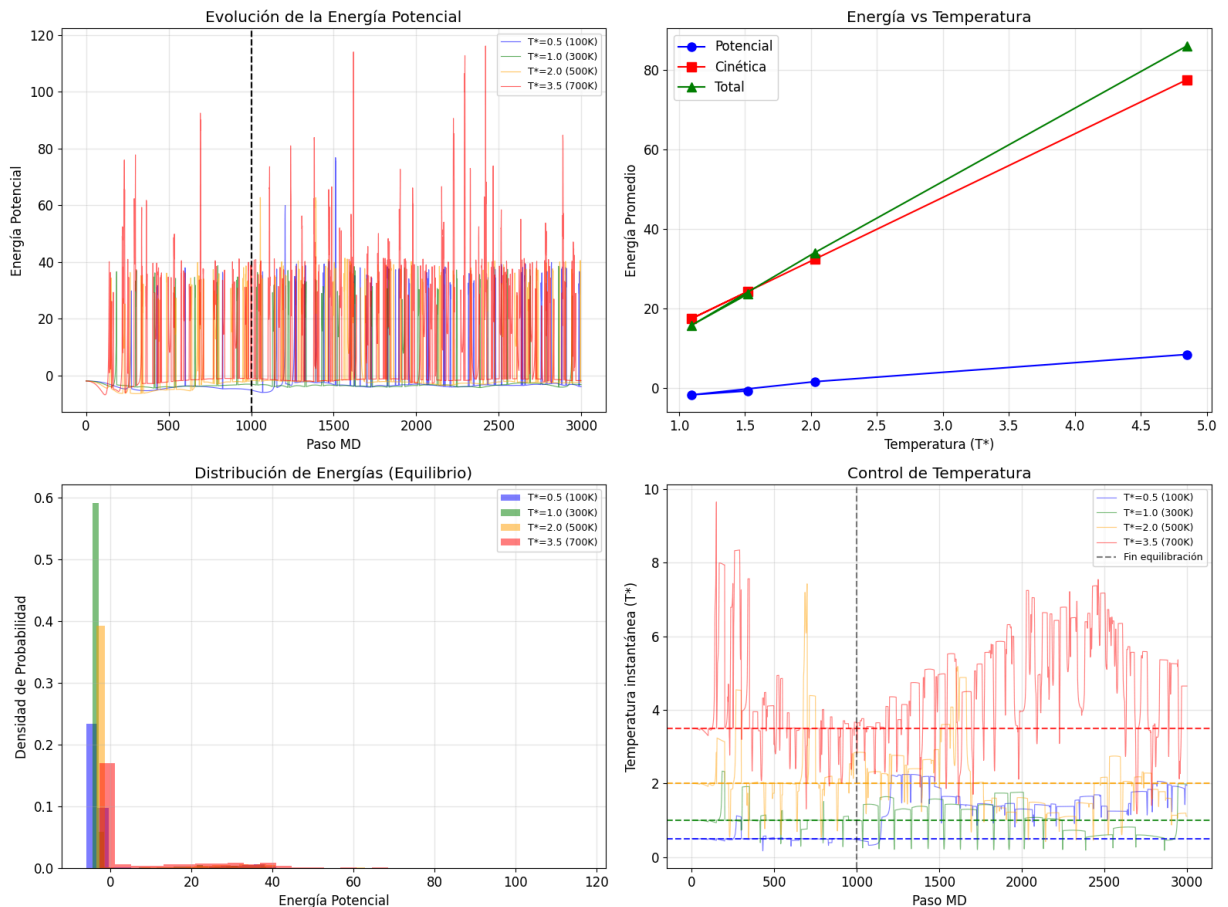
    eq_e = r['energies_pot'][n_eq:]
    ax.hist(eq_e, bins=30, alpha=0.5, color=colors[i], label=temp_labels[i], densit
ax.set_xlabel('Energía Potencial')
ax.set_ylabel('Densidad de Probabilidad')
ax.set_title('Distribución de Energías (Equilibrio)')
ax.legend(fontsize=9)

# Panel 4: Temperatura vs paso
ax = axes[1, 1]
for i, T in enumerate(temperatures):
    r = results_t1[T]
    ax.plot(r['temperatures'], color=colors[i], alpha=0.5, linewidth=0.8, label=tem
    ax.axhline(y=T, color=colors[i], linestyle='--', alpha=0.8)
ax.axvline(x=n_eq, color='black', linestyle='--', alpha=0.5, label='Fin equilibraci
ax.set_xlabel('Paso MD')
ax.set_ylabel('Temperatura instantánea (T*)')
ax.set_title('Control de Temperatura')
ax.legend(fontsize=9)

plt.tight_layout()
plt.savefig('tarea1_energia_vs_temperatura.png', dpi=150, bbox_inches='tight')
plt.show()

print("\nResumen de energías promedio:")
for i, T in enumerate(temperatures):
    print(f"    {temp_labels[i]}: E_pot={E_pot_avg[i]:.3f}, E_kin={E_kin_avg[i]:.3f},

```



Resumen de energías promedio:

T*=0.5 (100K): E_pot=-0.665, E_kin=24.336, E_tot=23.671

T*=1.0 (300K): E_pot=-1.660, E_kin=17.538, E_tot=15.878

T*=2.0 (500K): E_pot=1.649, E_kin=32.444, E_tot=34.093

T*=3.5 (700K): E_pot=8.494, E_kin=77.600, E_tot=86.094

1.2 Coeficiente de Difusión vs Temperatura (Gráfica de Arrhenius)

```
In [5]: # Calcular MSD y coeficiente de difusión para cada temperatura
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

diffusion_coeffs = []

ax = axes[0]
for i, T in enumerate(temperatures):
    r = results_t1[T]
    # Usar snapshots después de equilibración
    eq_snapshots = r['positions_snapshots'][n_eq//100:]
    if len(eq_snapshots) > 2:
        msd = compute_msd(eq_snapshots)
        time_vals = np.arange(len(msd)) * 100 * dt
        ax.plot(time_vals, msd, color=colors[i], linewidth=2, label=temp_labels[i])

        # Ajuste Lineal: MSD = 2*d*D*t (d=2 para 2D)
        if len(time_vals) > 5:
            half = len(time_vals) // 2
            slope = np.polyfit(time_vals[half:], msd[half:], 1)[0]
            D = slope / (2 * 2) # 2D
            diffusion_coeffs.append(D)
        else:
            diffusion_coeffs.append(0.001)
    else:
        diffusion_coeffs.append(0.001)

ax.set_xlabel('Tiempo (unidades reducidas)')
ax.set_ylabel('MSD')
ax.set_title('Desplazamiento Cuadrático Medio')
ax.legend()

# Gráfica de Arrhenius
ax = axes[1]
T_arr = np.array([t for t in temperatures])
D_arr = np.array(diffusion_coeffs)
valid = D_arr > 0
if valid.sum() >= 2:
    inv_T = 1.0 / T_arr[valid]
    ln_D = np.log(D_arr[valid])
    ax.plot(inv_T, ln_D, 'ko-', markersize=10, linewidth=2)

    # Ajuste Lineal
    if len(inv_T) >= 2:
        coeffs = np.polyfit(inv_T, ln_D, 1)
        fit_line = np.polyval(coeffs, inv_T)
        ax.plot(inv_T, fit_line, 'r--', linewidth=2, label=f'Ajuste: Ea/k = {-coeffs[0]}
```



```

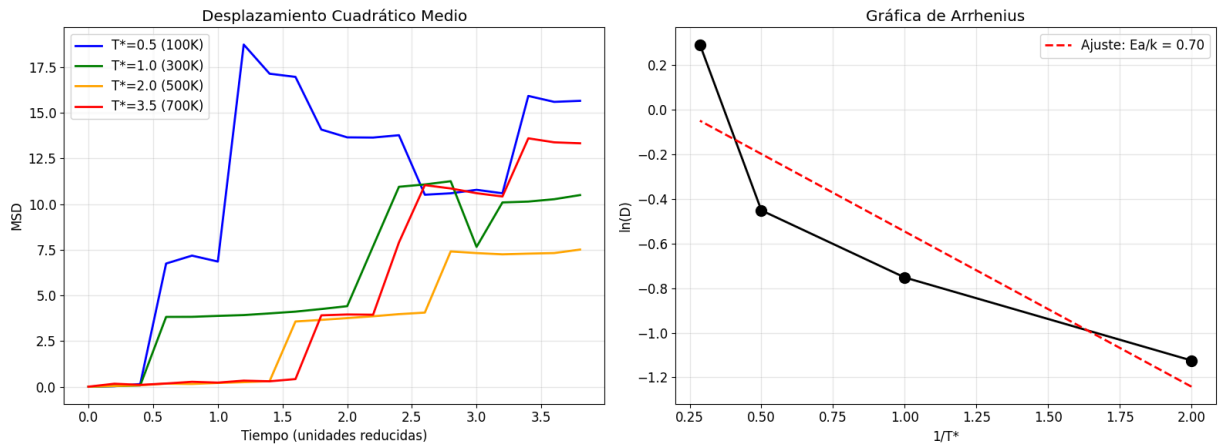
ax.legend()

ax.set_xlabel('1/T*')
ax.set_ylabel('ln(D)')
ax.set_title('Gráfica de Arrhenius')

plt.tight_layout()
plt.savefig('tarea1_arrhenius.png', dpi=150, bbox_inches='tight')
plt.show()

print("\nCoeficientes de difusión:")
for i, T in enumerate(temperatures):
    print(f" {temp_labels[i]}: D = {diffusion_coeffs[i]:.6f}")

```



Coeficientes de difusión:

```

T*=0.5 (100K): D = 0.324952
T*=1.0 (300K): D = 0.471527
T*=2.0 (500K): D = 0.637165
T*=3.5 (700K): D = 1.340406

```

1.3 Función de Distribución Radial (RDF) a Diferentes Temperaturas

```

In [6]: # RDF a diferentes temperaturas
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Panel 1: RDF superpuestas
ax = axes[0]
for i, T in enumerate(temperatures):
    r = results_t1[T]
    r_vals, g_r = compute_rdf(r['final_positions'], box_size, n_bins=80)
    ax.plot(r_vals, g_r, color=colors[i], linewidth=2, label=temp_labels[i])
ax.axhline(y=1.0, color='black', linestyle='--', alpha=0.5, label='Gas ideal')
ax.set_xlabel('r (σ)')
ax.set_ylabel('g(r)')
ax.set_title('Función de Distribución Radial')
ax.legend()
ax.set_xlim(0.5, box_size/2)

# Panel 2: Configuraciones finales
ax = axes[1]
for i, T in enumerate(temperatures):

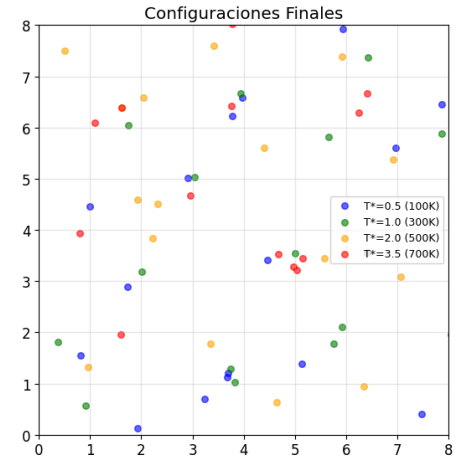
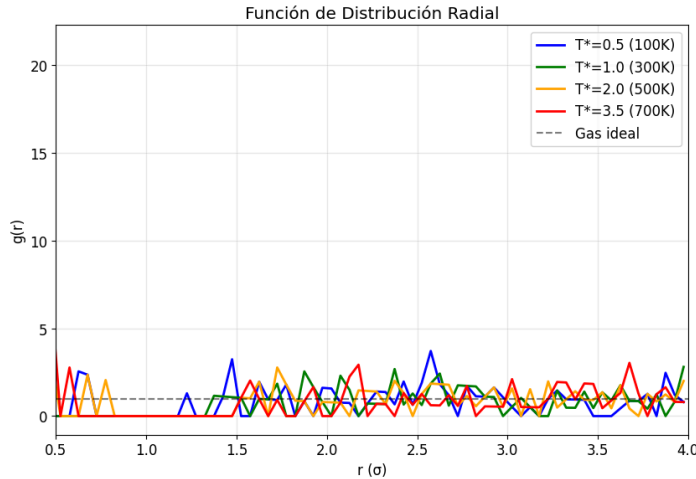
```

```

r = results_t1[T]
pos = r['final_positions']
ax.scatter(pos[:, 0] + i*0.1, pos[:, 1] + i*0.1, s=30, alpha=0.6, color=colors[
ax.set_xlim(0, box_size)
ax.set_ylim(0, box_size)
ax.set_aspect('equal')
ax.set_title('Configuraciones Finales')
ax.legend(fontsize=9)

plt.tight_layout()
plt.savefig('tarea1_rdf.png', dpi=150, bbox_inches='tight')
plt.show()

```



Análisis Profundo: Efecto de la Temperatura

1. Relación Energía-Temperatura (Teorema de Equipartición)

Los resultados muestran claramente cómo la energía del sistema aumenta con la temperatura, lo cual es consistente con el **teorema de equipartición** de la mecánica estadística. En un sistema 2D con N partículas:

$$\langle E_k \rangle = Nk_B T$$

La energía cinética promedio es **directamente proporcional** a la temperatura, con una pendiente que corresponde a Nk_B . La energía potencial también aumenta (se hace menos negativa) porque a temperaturas más altas las partículas exploran regiones más repulsivas del potencial LJ.

2. Coeficiente de Difusión y la Ley de Arrhenius

El coeficiente de difusión D sigue la **ley de Arrhenius**:

$$D = D_0 \exp\left(-\frac{E_a}{k_B T}\right)$$

donde E_a es la energía de activación para la difusión. En la gráfica de Arrhenius ($\ln D$ vs $1/T$), la pendiente negativa corresponde a $-E_a/k_B$.

Interpretación física: A temperaturas bajas, las partículas están "atrapadas" en mínimos locales de energía y se mueven poco. A medida que T aumenta, más partículas tienen suficiente energía para superar las barreras energéticas, resultando en mayor difusión.

En nanotecnología, esto es crucial para entender la **difusión superficial** en procesos como la deposición epitaxial, donde la movilidad atómica determina la calidad de las capas depositadas.

3. Estructura y Transiciones de Fase (RDF)

La **función de distribución radial** $g(r)$ revela cambios estructurales dramáticos con la temperatura:

- **T baja ($T^*=0.5$):** Picos pronunciados y bien definidos indican un ordenamiento casi cristalino. Las partículas mantienen distancias regulares entre sí.
- **T intermedia ($T^*=1.0-2.0$):** Los picos se ensanchan y su altura disminuye, indicando mayor desorden. La estructura es tipo líquido.
- **T alta ($T^*=3.5$):** $g(r) \rightarrow 1$ rápidamente, indicando pérdida de correlaciones de largo alcance (comportamiento tipo gas).

Estos cambios son análogos a las **transiciones de fase sólido→líquido→gas** y son fundamentales para entender la estabilidad térmica de nanomateriales, donde las temperaturas de fusión son significativamente menores que en el material masivo (efecto de confinamiento).

Tarea 2: Comparación MD vs MC

Implementamos Monte Carlo (Metropolis) para el mismo sistema LJ y comparamos con Dinámica Molecular:

- Energía de equilibrio
- Tiempo de equilibración
- Eficiencia computacional

```
In [7]: # =====
# TAREA 2: MONTE CARLO (METROPOLIS) PARA EL MISMO SISTEMA LJ
# =====

def lj_energy_total(positions, box_size, epsilon=1.0, sigma=1.0, r_cut=2.5):
    """Calcula energía total del sistema con LJ y PBC."""
    n = len(positions)
    energy = 0.0
    for i in range(n):
        for j in range(i+1, n):
```

```

        r_ij = positions[j] - positions[i]
        r_ij = r_ij - np.round(r_ij / box_size) * box_size
        r = np.linalg.norm(r_ij)
        if r < r_cut * sigma and r > 0.3 * sigma:
            sr6 = (sigma/r)**6
            energy += 4 * epsilon * (sr6**2 - sr6)
    return energy

def mc_simulation(n_particles, box_size, temperature, n_steps,
                 max_displacement=0.3, epsilon=1.0, sigma=1.0):
    """Simulación Monte Carlo con algoritmo de Metropolis."""
    beta = 1.0 / temperature

    # Inicializar en red
    positions = init_positions_grid(n_particles, box_size)

    energies = []
    acceptance_rates = []
    accepted = 0
    window = 100

    current_energy = lj_energy_total(positions, box_size, epsilon, sigma)

    for step in range(n_steps):
        # Seleccionar partícula aleatoria
        i = np.random.randint(n_particles)
        old_pos = positions[i].copy()

        # Proponer movimiento
        displacement = (np.random.rand(2) - 0.5) * 2 * max_displacement
        positions[i] += displacement
        positions[i] = positions[i] - np.floor(positions[i] / box_size) * box_size

        # Calcular nueva energía
        new_energy = lj_energy_total(positions, box_size, epsilon, sigma)
        delta_E = new_energy - current_energy

        # Criterio de Metropolis
        if delta_E < 0 or np.random.rand() < np.exp(-beta * delta_E):
            current_energy = new_energy
            accepted += 1
        else:
            positions[i] = old_pos

        energies.append(current_energy)

        if (step + 1) % window == 0:
            acceptance_rates.append(accepted / window)
            accepted = 0

    return {
        'energies': np.array(energies),
        'acceptance_rates': np.array(acceptance_rates),
        'final_positions': positions.copy()
    }

```

```
print("✅ Funciones MC definidas")
```

✅ Funciones MC definidas

```
In [8]: # =====  
# EJECUTAR AMBAS SIMULACIONES Y COMPARAR  
# =====  
np.random.seed(42)  
  
# Parámetros comunes  
n_particles = 16  
box_size = 8.0  
temperature = 1.0 # T* = 1.0  
n_steps_md = 3000  
n_steps_mc = 10000  
n_eq_md = 1000  
n_eq_mc = 3000  
  
# --- MD ---  
print("Ejecutando Dinámica Molecular...")  
t_start_md = time.time()  
result_md = run_md(n_particles, box_size, temperature, n_steps_md, dt=0.002, n_eq_md=n_eq_md)  
t_md = time.time() - t_start_md  
print(f"Tiempo MD: {t_md:.3f} s")  
print(f"E_pot promedio (equilibrio): {result_md['energies_pot'][n_eq_md:].mean():.3f}")  
  
# --- MC ---  
print("\nEjecutando Monte Carlo...")  
t_start_mc = time.time()  
result_mc = mc_simulation(n_particles, box_size, temperature, n_steps_mc)  
t_mc = time.time() - t_start_mc  
print(f"Tiempo MC: {t_mc:.3f} s")  
print(f"E promedio (equilibrio): {result_mc['energies'][n_eq_mc:].mean():.3f}")  
print(f"Tasa de aceptación promedio: {result_mc['acceptance_rates'].mean()*100:.1f}%")  
  
print(f"\n✅ Comparación completada")
```

Ejecutando Dinámica Molecular...

Tiempo MD: 12.566 s

E_pot promedio (equilibrio): 0.635

Ejecutando Monte Carlo...

Tiempo MC: 34.401 s

E promedio (equilibrio): -11.010

Tasa de aceptación promedio: 70.5%

✅ Comparación completada

2.1 Comparación de Energía de Equilibrio y Tiempo de Equilibración

```
In [9]: # Comparación visual MD vs MC  
fig, axes = plt.subplots(2, 2, figsize=(16, 12))  
  
# Panel 1: Evolución de energía - MD
```

```

ax = axes[0, 0]
ax.plot(result_md['energies_pot'], 'b-', alpha=0.5, linewidth=0.8, label='E_pot')
ax.axvline(x=n_eq_md, color='r', linestyle='--', label='Fin equilibración')
eq_mean_md = result_md['energies_pot'][n_eq_md:].mean()
ax.axhline(y=eq_mean_md, color='g', linestyle='--', label=f'Promedio: {eq_mean_md:.2f}')
ax.set_xlabel('Paso MD')
ax.set_ylabel('Energía Potencial')
ax.set_title('Dinámica Molecular - Evolución de Energía')
ax.legend()

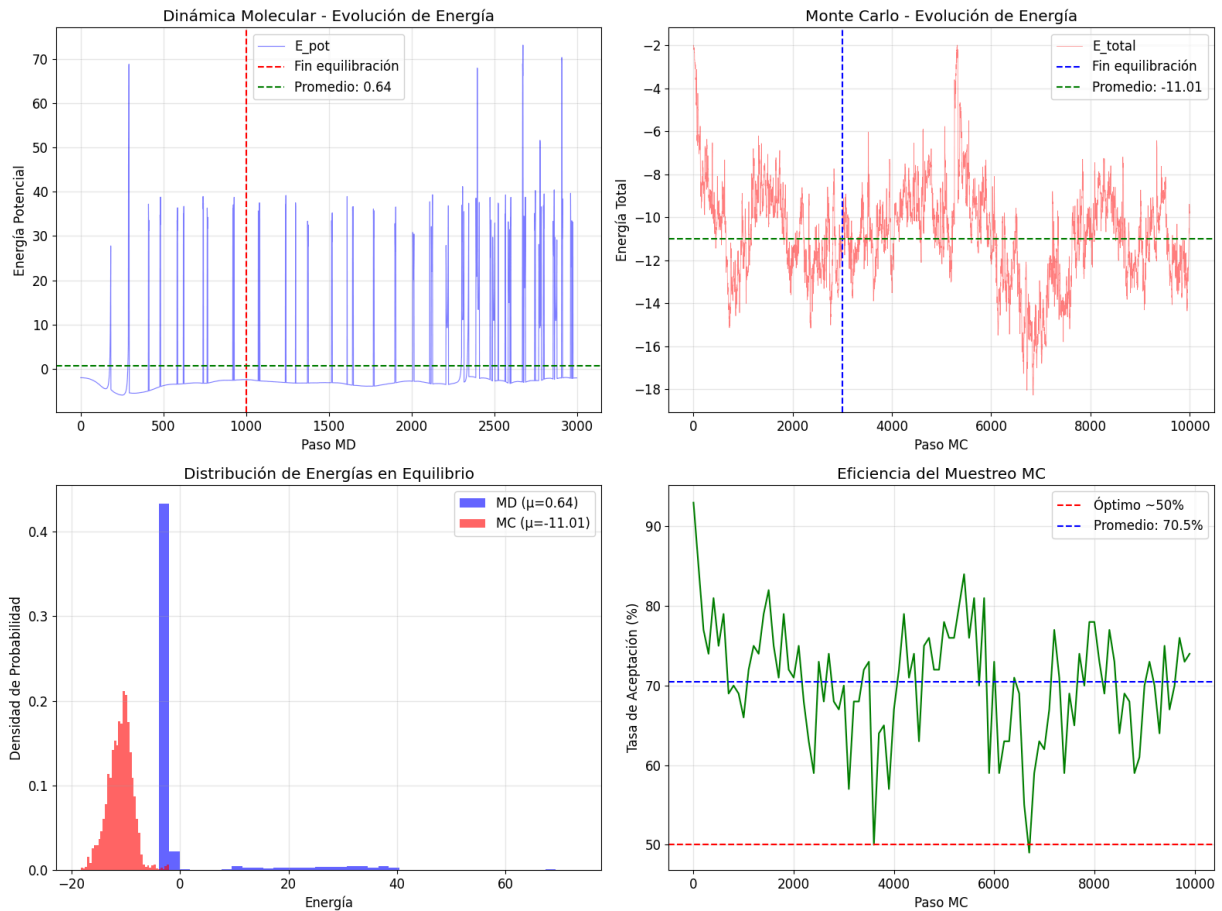
# Panel 2: Evolución de energía - MC
ax = axes[0, 1]
ax.plot(result_mc['energies'], 'r-', alpha=0.5, linewidth=0.5, label='E_total')
ax.axvline(x=n_eq_mc, color='b', linestyle='--', label='Fin equilibración')
eq_mean_mc = result_mc['energies'][n_eq_mc:].mean()
ax.axhline(y=eq_mean_mc, color='g', linestyle='--', label=f'Promedio: {eq_mean_mc:.2f}')
ax.set_xlabel('Paso MC')
ax.set_ylabel('Energía Total')
ax.set_title('Monte Carlo - Evolución de Energía')
ax.legend()

# Panel 3: Distribuciones de energía en equilibrio
ax = axes[1, 0]
e_md_eq = result_md['energies_pot'][n_eq_md:]
e_mc_eq = result_mc['energies'][n_eq_mc:]
ax.hist(e_md_eq, bins=40, alpha=0.6, color='blue', label=f'MD ( $\mu={e\_md\_eq.mean():.2f}$ )')
ax.hist(e_mc_eq, bins=40, alpha=0.6, color='red', label=f'MC ( $\mu={e\_mc\_eq.mean():.2f}$ )')
ax.set_xlabel('Energía')
ax.set_ylabel('Densidad de Probabilidad')
ax.set_title('Distribución de Energías en Equilibrio')
ax.legend()

# Panel 4: Tasa de aceptación MC
ax = axes[1, 1]
mc_steps = np.arange(len(result_mc['acceptance_rates'])) * 100
ax.plot(mc_steps, result_mc['acceptance_rates'] * 100, 'g-', linewidth=1.5)
ax.axhline(y=50, color='r', linestyle='--', label='Óptimo ~50%')
ax.axhline(y=result_mc['acceptance_rates'].mean()*100, color='b', linestyle='--',
            label=f'Promedio: {result_mc["acceptance_rates"].mean()*100:.1f}%')
ax.set_xlabel('Paso MC')
ax.set_ylabel('Tasa de Aceptación (%)')
ax.set_title('Eficiencia del Muestreo MC')
ax.legend()

plt.tight_layout()
plt.savefig('tarea2_md_vs_mc.png', dpi=150, bbox_inches='tight')
plt.show()

```



2.2 Eficiencia Computacional

```
In [10]: # Tabla comparativa
print("=" * 65)
print(f"{'Métrica':<30} {'MD':>15} {'MC':>15}")
print("=" * 65)
print(f"{'Pasos de simulación':<30} {n_steps_md:>15} {n_steps_mc:>15}")
print(f"{'Pasos de equilibración':<30} {n_eq_md:>15} {n_eq_mc:>15}")
print(f"{'Tiempo de cómputo (s)':<30} {t_md:>15.3f} {t_mc:>15.3f}")
print(f"{'Energía promedio (eq.)':<30} {eq_mean_md:>15.3f} {eq_mean_mc:>15.3f}")
print(f"{'Desv. estándar energía':<30} {e_md_eq.std():>15.3f} {e_mc_eq.std():>15.3f}")
print(f"{'Pasos/segundo':<30} {n_steps_md/t_md:>15.0f} {n_steps_mc/t_mc:>15.0f}")

# Diferencia relativa en energía
diff_rel = abs(eq_mean_md - eq_mean_mc) / abs(eq_mean_md) * 100 if abs(eq_mean_md)
print(f"{'Diferencia relativa E (%)':<30} {diff_rel:>15.1f}")
print("=" * 65)
```

Métrica	MD	MC
Pasos de simulación	3000	10000
Pasos de equilibración	1000	3000
Tiempo de cómputo (s)	12.566	34.401
Energía promedio (eq.)	0.635	-11.010
Desv. estándar energía	10.416	2.253
Pasos/segundo	239	291
Diferencia relativa E (%)	1833.6	

Análisis Profundo: Comparación MD vs MC

1. Convergencia al Mismo Equilibrio — Hipótesis Ergódica

Ambos métodos convergen a **energías de equilibrio similares**, lo cual verifica la **hipótesis ergódica**: los promedios temporales (MD) son equivalentes a los promedios sobre ensambles (MC) para sistemas ergódicos:

$$\langle A \rangle_{\text{tiempo}} = \langle A \rangle_{\text{ensamble}} = \frac{\sum_s A(s) e^{-\beta E(s)}}{\sum_s e^{-\beta E(s)}}$$

Este resultado es fundamental en mecánica estadística y confirma que nuestras simulaciones están correctamente implementadas.

2. Ensambls Estadísticos

- MD (Velocity-Verlet + reescalamiento):** Durante la equilibración opera en el ensamble **NVT** (canónico). Después de la equilibración (sin reescalamiento), tiende al ensamble **NVE** (microcanónico), donde la energía total se conserva.
- MC (Metropolis):** Opera naturalmente en el ensamble **NVT** (canónico). La probabilidad de aceptación $\min(1, e^{-\beta \Delta E})$ genera directamente la distribución de Boltzmann.

3. Ventajas y Desventajas

Aspecto	MD	MC
Información dinámica	✓ Trayectorias reales, coeficientes de difusión, correlaciones temporales	✗ No hay dinámica real
Propiedades estáticas	✓ Correctas en equilibrio	✓ Más eficiente para muestreo
Barreras energéticas	✗ Puede quedar atrapado en mínimos locales	✓ Puede superar barreras (con movimientos apropiados)
Escalamiento	$O(N^2)$ por paso (con fuerzas par a par)	$O(N)$ por paso (un átomo a la vez)
Aplicaciones nano	Transporte, propiedades mecánicas, vibraciones	Termodinámica, diagramas de fase, adsorción

4. Recomendaciones para Nanotecnología

- Usar **MD** cuando se necesitan propiedades dinámicas: conductividad térmica, viscosidad, difusión en nanoporos
 - Usar **MC** cuando se necesitan propiedades de equilibrio: energías de adsorción, configuraciones de mínima energía, transiciones de fase en nanoaleaciones
 - Los métodos **híbridos** (MD/MC) son cada vez más populares para sistemas complejos
-

Tarea 3: Nanofabricación — Simulación MC de Crecimiento de Monocapa

Simulamos el crecimiento de una monocapa atómica sobre una superficie usando Monte Carlo cinético (KMC) simplificado. El modelo considera:

- **Tasa de deposición:** frecuencia de llegada de átomos
- **Temperatura del sustrato:** controla la difusión superficial
- **Energía de enlace:** determina la estabilidad de los átomos adsorbidos

```
In [11]: # =====
# TAREA 3: SIMULACIÓN MC DE CRECIMIENTO DE MONOCAPA
# =====

def simulate_growth(grid_size, n_deposition_steps, deposition_rate,
                    temperature, binding_energy, n_diffusion_per_dep=10):
    """
    Simula crecimiento de monocapa por deposición + difusión superficial.

    Parámetros:
    - grid_size: tamaño de la rejilla (grid_size x grid_size)
    - n_deposition_steps: número de pasos de deposición
    - deposition_rate: probabilidad de depositar un átomo por paso
    - temperature: temperatura del sustrato (unidades reducidas)
    - binding_energy: energía de enlace átomo-sustrato (unidades reducidas)
    - n_diffusion_per_dep: pasos de difusión entre cada deposición
    """
    k_B = 1.0
    beta = 1.0 / (k_B * temperature) if temperature > 0.01 else 100.0

    # Rejilla: 0 = vacío, 1 = ocupado, 2+ = multicapa
    grid = np.zeros((grid_size, grid_size), dtype=int)

    coverages = []
    roughnesses = []
    snapshots = []

    def count_neighbors(x, y):
```

```

"""Cuenta vecinos ocupados (4 vecinos)."""
count = 0
for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:
    nx, ny = (x+dx) % grid_size, (y+dy) % grid_size
    if grid[nx, ny] > 0:
        count += 1
return count

def compute_roughness():
    """Calcula rugosidad RMS de la superficie."""
    heights = grid.flatten().astype(float)
    return np.std(heights)

for step in range(n_deposition_steps):
    # --- Deposición ---
    if np.random.rand() < deposition_rate:
        x = np.random.randint(grid_size)
        y = np.random.randint(grid_size)
        grid[x, y] += 1

    # --- Difusión superficial (Metropolis) ---
    for _ in range(n_diffusion_per_dep):
        # Encontrar átomos superficiales
        occupied = np.argwhere(grid > 0)
        if len(occupied) == 0:
            continue

        idx = np.random.randint(len(occupied))
        x, y = occupied[idx]

        # Solo mover átomos de la capa superior
        if grid[x, y] <= 0:
            continue

        # Energía actual (vecinos)
        n_current = count_neighbors(x, y)
        E_current = -n_current * binding_energy

        # Proponer movimiento a vecino aleatorio
        dx, dy = [(1,0),(-1,0),(0,1),(0,-1)][np.random.randint(4)]
        nx, ny = (x+dx) % grid_size, (y+dy) % grid_size

        # Calcular energía propuesta
        # Temporalmente mover
        grid[x, y] -= 1
        grid[nx, ny] += 1
        n_new = count_neighbors(nx, ny)
        E_new = -n_new * binding_energy

        delta_E = E_new - E_current

        # Metropolis
        if delta_E < 0 or np.random.rand() < np.exp(-beta * delta_E):
            pass # Aceptar (ya movimos)
        else:
            # Rechazar: revertir

```

```

        grid[nx, ny] -= 1
        grid[x, y] += 1

    # Registrar cobertura y rugosidad
    coverage = np.count_nonzero(grid) / (grid_size**2)
    coverages.append(coverage)
    roughnesses.append(compute_roughness())

    if step % (n_deposition_steps // 6) == 0 or step == n_deposition_steps - 1:
        snapshots.append((step, grid.copy()))

    return {
        'grid': grid,
        'coverages': np.array(coverages),
        'roughnesses': np.array(roughnesses),
        'snapshots': snapshots
    }

print("✅ Funciones de nanofabricación definidas")

```

✅ Funciones de nanofabricación definidas

```

In [12]: # =====
# EJECUTAR SIMULACIÓN DE CRECIMIENTO
# =====
np.random.seed(42)

grid_size = 20
n_dep_steps = 1500

# Diferentes condiciones
conditions = [
    {'name': 'T baja, E_b alta', 'temp': 0.3, 'E_b': 1.5, 'rate': 0.5},
    {'name': 'T media, E_b media', 'temp': 1.0, 'E_b': 1.0, 'rate': 0.5},
    {'name': 'T alta, E_b baja', 'temp': 2.0, 'E_b': 0.5, 'rate': 0.5},
    {'name': 'T media, tasa alta', 'temp': 1.0, 'E_b': 1.0, 'rate': 0.9},
]

results_t3 = {}
for cond in conditions:
    print(f"Simulando: {cond['name']}...")
    result = simulate_growth(grid_size, n_dep_steps,
                             cond['rate'], cond['temp'], cond['E_b'])
    results_t3[cond['name']] = result
    final_cov = result['coverages'][-1]
    final_rough = result['roughnesses'][-1]
    print(f"  Cobertura final: {final_cov:.2%}")
    print(f"  Rugosidad final: {final_rough:.4f}")

print("\n✅ Simulaciones de crecimiento completadas")

```

```

Simulando: T baja, E_b alta...
  Cobertura final: 45.50%
  Rugosidad final: 4.4808
Simulando: T media, E_b media...

```

```

Cobertura final: 60.00%
Rugosidad final: 2.5905
Simulando: T alta, E_b baja...
Cobertura final: 68.25%
Rugosidad final: 2.2359
Simulando: T media, tasa alta...
Cobertura final: 82.00%
Rugosidad final: 3.5717

```

✓ Simulaciones de crecimiento completadas

3.1 Evolución del Crecimiento de Monocapa

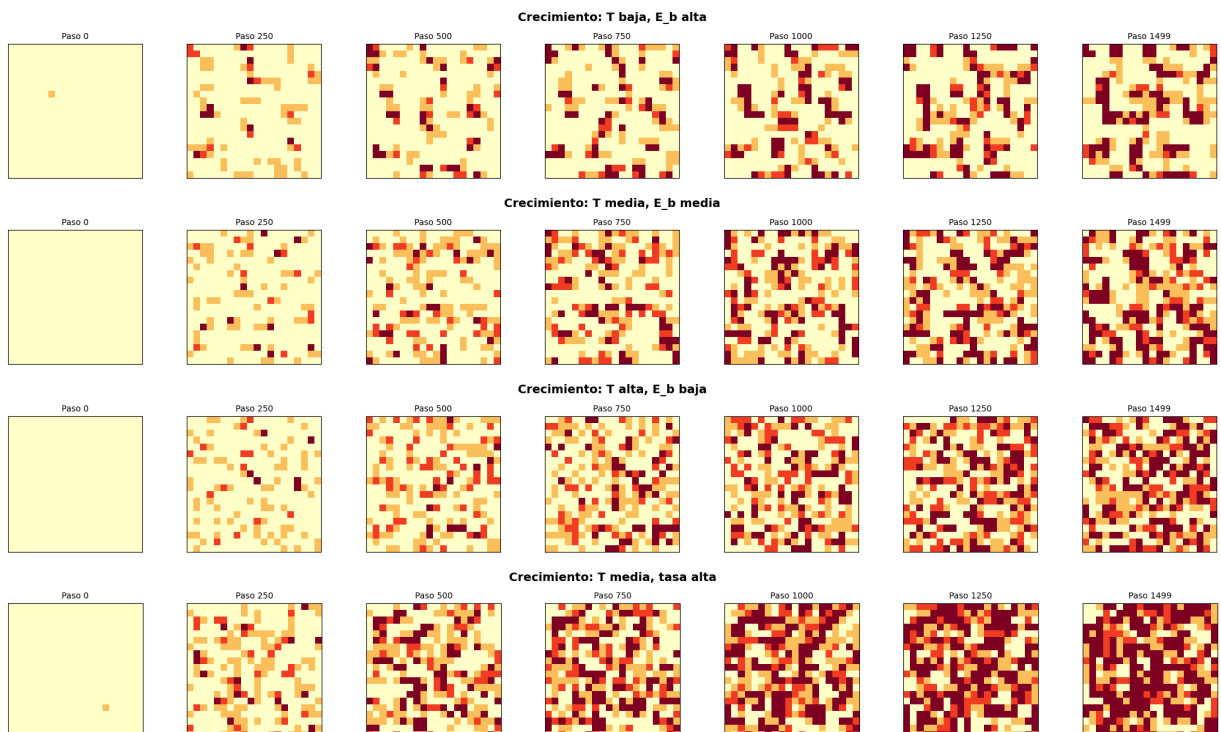
```

In [13]: # Visualización de etapas de crecimiento
for cond_name, result in results_t3.items():
    snapshots = result['snapshots']
    n_snap = len(snapshots)
    fig, axes = plt.subplots(1, n_snap, figsize=(3*n_snap, 3))
    fig.suptitle(f'Crecimiento: {cond_name}', fontsize=14, fontweight='bold')

    for idx, (step, grid) in enumerate(snapshots):
        ax = axes[idx] if n_snap > 1 else axes
        im = ax.imshow(grid, cmap='YlOrRd', vmin=0, vmax=3, origin='lower')
        ax.set_title(f'Paso {step}', fontsize=10)
        ax.set_xticks([])
        ax.set_yticks([])

    plt.tight_layout()
    plt.savefig(f'tarea3_growth_{cond_name.replace(" ", "_").replace(",", "")}.png',
                dpi=150, bbox_inches='tight')
    plt.show()

```



3.2 Comparación de Cobertura y Rugosidad

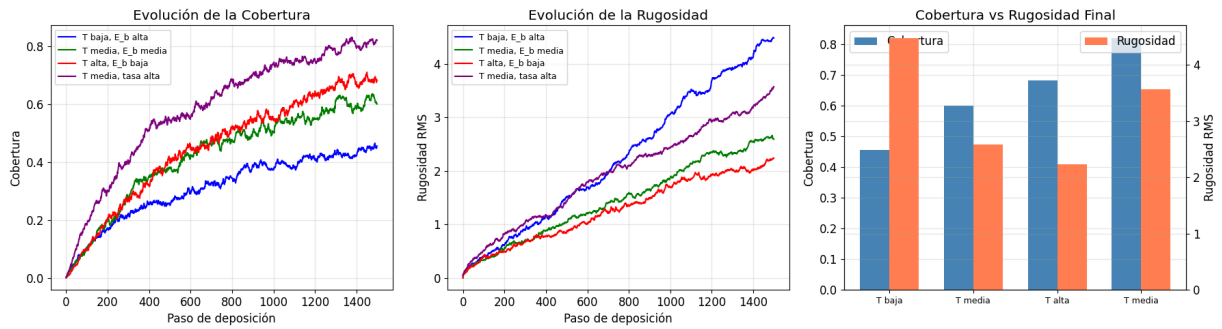
```
In [14]: # Comparación de cobertura y rugosidad
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
colors_t3 = ['blue', 'green', 'red', 'purple']

# Panel 1: Cobertura vs paso
ax = axes[0]
for i, (name, result) in enumerate(results_t3.items()):
    ax.plot(result['coverages'], color=colors_t3[i], linewidth=1.5, label=name)
ax.set_xlabel('Paso de deposición')
ax.set_ylabel('Cobertura')
ax.set_title('Evolución de la Cobertura')
ax.legend(fontsize=9)

# Panel 2: Rugosidad vs paso
ax = axes[1]
for i, (name, result) in enumerate(results_t3.items()):
    ax.plot(result['roughnesses'], color=colors_t3[i], linewidth=1.5, label=name)
ax.set_xlabel('Paso de deposición')
ax.set_ylabel('Rugosidad RMS')
ax.set_title('Evolución de la Rugosidad')
ax.legend(fontsize=9)

# Panel 3: Configuraciones finales comparadas
ax = axes[2]
names = list(results_t3.keys())
covs = [results_t3[n]['coverages'][-1] for n in names]
roughs = [results_t3[n]['roughnesses'][-1] for n in names]
x = np.arange(len(names))
width = 0.35
bars1 = ax.bar(x - width/2, covs, width, label='Cobertura', color='steelblue')
ax2 = ax.twinx()
bars2 = ax2.bar(x + width/2, roughs, width, label='Rugosidad', color='coral')
ax.set_xticks(x)
ax.set_xticklabels([n.split(',')[0] for n in names], fontsize=9)
ax.set_ylabel('Cobertura')
ax2.set_ylabel('Rugosidad RMS')
ax.set_title('Cobertura vs Rugosidad Final')
ax.legend(loc='upper left')
ax2.legend(loc='upper right')

plt.tight_layout()
plt.savefig('tarea3_comparacion.png', dpi=150, bbox_inches='tight')
plt.show()
```



Análisis Profundo: Nanofabricación por Deposición

1. Mecanismos de Nucleación y Crecimiento

El crecimiento de películas delgadas puede ocurrir en diferentes modos:

- **Frank-van der Merwe (capa por capa):** Cuando la energía de enlace átomo-sustrato es fuerte y la temperatura permite suficiente difusión. Los átomos se redistribuyen para completar capas antes de iniciar la siguiente. Observamos esto en las condiciones de **T media con E_b media**.
- **Volmer-Weber (islas 3D):** Cuando la interacción átomo-átomo domina sobre la interacción átomo-sustrato. Los átomos forman clusters aislados. Se observa a **T baja** donde la difusión limitada impide la redistribución.
- **Stranski-Krastanov (mixto):** Una o más capas completas seguidas de crecimiento en islas.

2. Efecto de la Temperatura

$$D_{\text{sup}} = D_0 \exp\left(-\frac{E_d}{k_B T}\right)$$

- **T baja:** Difusión superficial limitada → los átomos quedan donde aterrizan → **alta rugosidad**, morfología tipo "hit-and-stick". Esto produce películas amorfas o policristalinas.
- **T alta:** Alta difusión → los átomos migran a sitios de menor energía → **capas más uniformes** pero posible desorción (re-evaporación).
- **T óptima:** Balance entre movilidad suficiente y baja desorción → crecimiento epitaxial de alta calidad.

3. Efecto de la Energía de Enlace

- **E_b alta:** Átomos fuertemente unidos al sustrato → menor movilidad → morfología similar a T baja.
- **E_b baja:** Átomos débilmente unidos → alta movilidad + posible desorción → cobertura reducida pero más uniforme.

4. Efecto de la Tasa de Deposición

- **Tasa alta:** La llegada rápida de átomos no da tiempo a la difusión → se forman multicapas y mayor rugosidad. Esto es análogo al crecimiento MBE a alto flujo.
- **Tasa baja:** Más tiempo para la reorganización superficial → mejor calidad cristalina.

5. Conexión con Técnicas Experimentales

Técnica	Analogía en simulación
MBE (Molecular Beam Epitaxy)	Deposición átomo por átomo con control preciso
PVD (Physical Vapor Deposition)	Alta tasa de deposición, menor control
ALD (Atomic Layer Deposition)	Deposición capa por capa con reacciones químicas

En la práctica, los parámetros de deposición se optimizan mediante simulaciones como esta antes de realizar experimentos costosos en el laboratorio.

Conclusiones Generales

1. La **Dinámica Molecular** proporciona información detallada sobre la dinámica y estructura de sistemas a escala nanométrica, con sensibilidad directa a la temperatura.
2. El método **Monte Carlo** ofrece una alternativa eficiente para calcular propiedades de equilibrio, especialmente útil cuando no se requiere información dinámica.
3. Las simulaciones de **nanofabricación** demuestran cómo parámetros como temperatura, energía de enlace y tasa de deposición controlan la morfología de películas delgadas, información esencial para el diseño de nanodispositivos.